CAAM 37711 — Homework 2 Solutions (Fall 2026)

Filippo Delzanno

November 6, 2026

Handwritten solutions

The solutions to Problem 1(a), (b), and (d) are handwritten and attached. The rest of the results are numerical, from evaluating the code submitted with this assignment.

Filippo Delzanno

Problem Set #2 (CAAM 37711)

(1) a - We define a neural architecture $g : \mathbb{R}^p \to \mathbb{R}^2$, where the input to g is a p-dimensional feature vector $x \in \mathbb{R}^p$. Thus, the network structure is:

$$g(x) = \begin{pmatrix} g_1(x) \\ g_2(x) \end{pmatrix}, \quad \text{where } g_1, g_2 \in \mathbb{R} \text{ and } g \in \mathbb{R}^2.$$

- $f_\mu(x)$ is used to approximate $\mathbb{E}[Y|X] \in \mathbb{R}$. Since $f_\mu(x) \in \mathbb{R}$, we do not need to modify the data type of $f_\mu(x)$, and can just take the last link function to be the identity:

$$\boxed{f_\mu(x) = g_1(x)}$$

- $f_\sigma(x)$ is used to approximate $\sqrt{\mathbb{V}ar(Y|X)} \in \mathbb{R}^+$. Since the standard deviation must be positive but $g_2(x) \in \mathbb{R}$, we need to choose a link function that maps $\mathbb{R} \to \mathbb{R}^+$. This is naturally achieved through an exponential link function:

$$\boxed{f_\sigma(x) = e^{g_2(x)}}$$

b - We model data of the form in (a) as $Y|X \sim \mathcal{N}(f_\mu(x), f_\sigma(x)^2)$. The natural loss function for this model is the negative log-likelihood. For the Gaussian pdf

$$\mathbb{P}(y_i|x_i) = \frac{1}{\sqrt{2\pi} f_\sigma(x_i)} e^{-\frac{(y_i - f_\mu(x_i))^2}{2 f_\sigma(x_i)^2}},$$

the likelihood of the dataset (assuming the observations are independent) is given by

$$L = \prod_{i=1}^n \mathbb{P}(y_i|x_i) \longrightarrow \mathcal{L} = -\frac{1}{n}\log L = -\frac{1}{n}\sum_{i=1}^n \log\left[\mathbb{P}(y_i|x_i)\right],$$

where in the last step, we average on n samples to make the loss a per-sample average. Expanding the pdf above:

$$\mathcal{L} = -\frac{1}{n}\sum_{i=1}^n \left(\log\left[\frac{1}{\sqrt{2\pi} f_\sigma(x_i)}\right] + \log\left[e^{-\frac{(y_i - f_\mu(x_i))^2}{2 f_\sigma(x_i)^2}}\right] \right) = -\frac{1}{n}\sum_{i=1}^n \left[-\frac{1}{2}\log(2\pi) - \log(f_\sigma(x_i)) - \frac{(y_i - f_\mu(x_i))^2}{2 f_\sigma(x_i)^2} \right]$$

$$\longrightarrow \mathcal{L} = \frac{1}{n}\sum_{i=1}^n \left[\frac{1}{2}\log(2\pi) + \log(f_\sigma(x_i)) + \frac{(y_i - f_\mu(x_i))^2}{2 f_\sigma(x_i)^2} \right]$$

* we can drop the $\frac{1}{2}\log(2\pi)$ term, since it will not enter in the minimization procedure

Thus, the loss function we can use for this data is:

$$\boxed{\mathcal{L} = \frac{1}{n}\sum_{i=1}^n \left[\log(f_\sigma(x_i)) + \frac{(y_i - f_\mu(x_i))^2}{2 f_\sigma(x_i)^2} \right] = \frac{1}{n}\sum_{i=1}^n \left[g_2(x_i) + \frac{1}{2}(y_i - g_1(x_i))^2 e^{-2g_2(x_i)} \right]}$$

penalizes inflated variance — noisy points count less

Filippo Delzanno

Problem Set #2 (CAAM 37711)

(1) d - The model we have implemented suggests that, given the feature X of a trip:

$$\log Y \mid X = x \sim \mathcal{N}(f_\mu(x), f_\sigma(x)^2),$$

where $f_\mu(x)$ is the predicted mean and $f_\sigma(x)^2$ is the predicted variance. The statement, "what is the probability that a trip will last < 45 minutes" is translated into the language of our model as:

$$\mathbb{P}(Y < 45 \mid X = x) = \mathbb{P}(\log Y < \log 45 \mid X = x) = \Phi\left[\frac{\log 45 - f_\mu(x)}{f_\sigma(x)}\right],$$

where Φ is the CDF of the normal distribution.

All code for this assignment is in the GitHub repository `https://github.com/fdelzanno/CAAM-37711`, under the `homework-2/` directory. See `README.md` for setup and run instructions.

Problem 1

(c) Model, baselines, and results

We choose to model log(trip duration) to ensure that probability is never assigned to trips with negative durations and split the data 70/15/15 into training, validation, and test sets. We also standardized and clipped the features using training-set statistics. The network is a ReLU MLP with hidden layers (128, 128, 64) where the first output is the mean $\mu(x)$, and the second is passed through $\sigma(x) = e^{g_2(x)}$ (to keep the standard deviation positive). We train by minimizing the Gaussian negative log-likelihood from (b) and use mini-batch SGD with gradient-norm clipping, a learning-rate scheduler that reduces the rate on plateaus, and early stopping on validation loss. The model is compared against four baselines: a constant model, which predicts the training mean and variance, along with linear regression, gradient boosting, and an MLP trained with MSE loss, each predicting a mean (with one constant variance taken from their validation residuals). All models are evaluated on the held-out test set using NLL, RMSE, and MAE, and the (empirical) coverage of the 68% and 95% intervals (used to determine whether the model's uncertainty estimates are accurate).

Model	NLL	RMSE	MAE	MAE (min)	Cover 68%	Cover 95%
Constant	1.172	0.781	0.615	5.57	0.697	0.964
Linear regression	0.885	0.586	0.438	4.63	0.727	0.957
Gradient boosting	0.448	0.379	0.259	2.30	0.787	0.956
MLP (MSE loss)	0.461	0.383	0.264	2.37	0.784	0.955
MLP (Gaussian NLL)	0.239	0.380	0.258	2.29	0.702	0.948

Table 1: Test-set metrics for each model on log(trip duration). NLL is the mean Gaussian negative log-likelihood, RMSE and MAE are in log-minutes, MAE (min) is in minutes, and Cover q% is the fraction of test points within the central q% predictive interval.

The Gaussian-NLL MLP and gradient boosting are the best predictors, with an MAE of about 2.3 minutes versus 4.6 for linear regression and 5.6 for the constant model. The MSE-trained MLP is slightly worse. The nonlinear models are therefore capturing structure that a linear fit cannot. The Gaussian-NLL MLP's advantage shows up in the NLL, since its MAE is essentially tied with gradient boosting. This is likely because it learns an input-dependent $\sigma(x)$, which the baselines cannot learn. The coverage results support this: the 68% intervals of linear regression, gradient boosting, and the MSE-trained MLP cover 73–79% of test points — so they are too wide for typical trips, likely because a few large residuals inflate the constant variance — while the Gaussian-NLL MLP covers 70.2%, close to the expected 68%.

(e) Evaluation of the predicted probabilities

For each model, we computed the predicted probability that a test trip takes under 45 minutes (from the formula in part (d)) and compared it to the observed 0/1 label. As a proper scoring rule, we report both the Brier score — the mean squared difference between the predicted probability and the outcome — and the log-loss — the mean negative log-likelihood of the observed labels.

Accuracy is obtained by predicting "under 45 minutes" whenever the probability is at least 0.5 and comparing it to the true label. Since almost every trip is short (99.55% take under 45 minutes), always predicting "under 45 minutes" already achieves 99.55% accuracy. This serves as the reference point.

Model	Brier score	Log-loss	Accuracy
Always predict <45 min	0.0045	–	0.9955
Constant	0.0046	0.0327	0.9955
Linear regression	0.0122	0.0453	0.9838
Gradient boosting	0.0029	0.0160	0.9968
MLP (MSE loss)	0.0033	0.0176	0.9965
MLP (Gaussian NLL)	0.0028	0.0170	0.9966

Table 2: Test-set Brier score, log-loss, and accuracy for $P(\text{trip} < 45 \text{ min})$.

We see that accuracy is nearly saturated and cannot effectively distinguish the models. The constant model only ties the trivial predictor, and linear regression does worse. The nonlinear models, however, gain only about a tenth of a percentage point. The Brier score and log-loss are more useful: both rank linear regression worst, with gradient boosting, the MLP with MSE loss, and the Gaussian-NLL MLP far ahead of the constant and linear baselines. The Gaussian-NLL MLP has the lowest Brier score, although gradient boosting has the lowest log-loss and the highest accuracy. Since the differences among these three are very small, it is difficult to say with any certainty which model performs best.

Problem 2

(a) Cardinality of the features and data splits

Feature	Unique values	Median rows	Min rows	Max rows	Values in ≤ 5 rows
StockCode	3,665	49	1	2,035	548
CustomerID	4,339	41	1	7,847	300

Table 3: Cardinality of the categorical features and the number of rows in which each unique value appears.

From the table above, we can comment on the cardinality of the features. StockCode has 3,665 unique values and CustomerID has 4,339, across 397,924 rows. Both features are high-cardinality and demonstrate uneven category sizes: the median category appears in only 49 rows for StockCode and 41 for CustomerID, while the largest appears in 2,035 and 7,847 rows, respectively. 548 stock codes (about 15%) and 300 customers (about 7%) appear in five or fewer rows. We could perform a one-hot encoding, but the large number of categories makes it impractical. It would create 8,004 columns (3,665 + 4,339) and require a dense float64 matrix of about 25.5 GB, almost entirely comprised of zeros. Even with a sparse representation, a model would have to learn a separate weight for each category, which becomes an issue for rare categories where there is almost no data to learn from. This suggests encodings that compress each category into a few representative numbers.

We randomly split the data into training, validation, and test sets (70/15/15%). Note that this

Feature	Validation rows	Test rows
StockCode	30	48
CustomerID	19	23

Table 4: Number of validation and test rows whose category never appears in the training set.

inevitably leads to some rare categories appearing only in validation or test. We quantify this in Table 4: 30/48 StockCode rows and 19/23 CustomerID rows in the validation/test sets (respectively) have a category not seen in training. Our model will still need to produce a prediction for these rows. The validation set is used for early stopping when training the neural network models in parts (c) and (d), and the test set is held out for final evaluation.

(b)–(d) Target encoding, embeddings, and the combined model

Model	Train deviance	Val. deviance	Test deviance	Test MAE	Test RMSE
Baseline	36.23	30.30	25.86	12.63	37.06
Target Encoding	13.29	10.21	8.38	7.14	28.11
Embeddings	9.24	8.26	6.81	6.22	27.63
Combined	10.54	8.57	6.98	6.41	27.00

Table 5: Poisson deviance (train, validation, test), and test MAE and RMSE for the mean baseline, the target-encoding model, the embedding network, and the combined network.

(b) To implement target encoding, we replace each category with its (smoothed) mean `Quantity` from the training set. Since `Quantity` is a positive integer (count) that is heavy-tailed, we fit a gradient-boosting model with a Poisson loss. We also cap the training target at the 99.9th percentile to stop a few huge counts from destabilizing the fit. This result is much better than the baseline that predicts the training mean: test deviance (the Poisson-loss counterpart of mean squared error) falls from 25.86 to 8.38, MAE from 12.63 to 7.14, and RMSE from 37.06 to 28.11.

(c) To develop an embedding neural network, we first need to assign every stock code and every customer a list of 16 numbers. These numbers are adjusted by the network during training. Then, the two learned embeddings are fed into a one-hidden-layer network (64 units) whose output is the log of the predicted `Quantity`. We train this network with Poisson loss, and find that it beats target encoding on every metric: test deviance is 6.81 vs. 8.38, MAE is 6.22 vs. 7.14, and RMSE is 27.63 vs. 28.11. This difference likely stems from the flexibility of the embedding. By tuning directly on the training data, it is likely to improve the final prediction compared to target encoding, which must summarize each category by one fixed number.

(d) To develop the combined model, we feed the embeddings and the two target-encoded features into the same neural network. We find that it beats target encoding (test deviance 6.98 vs. 8.38) but not the embeddings alone. It is also slightly worse on deviance (6.98 vs. 6.81) and MAE (6.41 vs. 6.22), but slightly better on RMSE (27.00 vs. 27.63). To identify whether this was a true shortcoming of the combined model, we trained the same network across five seeds (i.e., across 5 random initializations with the data split held fixed), with those results in Table 6. We found that the paired difference (Combined − Embeddings) is 0.05 ± 0.11 in test deviance and 0.10 ± 0.11 in test MAE. Since both are within noise, we see that adding target encoding to the embedding has

Model	Val. deviance	Test deviance	Test MAE
Embeddings	8.44 ± 0.11	6.99 ± 0.12	6.42 ± 0.17
Combined	8.62 ± 0.05	7.04 ± 0.05	6.52 ± 0.10
Combined − Embeddings	0.17 ± 0.11	0.05 ± 0.11	0.10 ± 0.11

Table 6: Embedding and combined networks refit under five random seeds (the seed used above plus four more with the same data split). Entries are mean ± standard deviation across seeds. The last row is the paired per-seed difference (Combined − Embeddings).

no clear advantage. This seems plausible, given that the embeddings can already learn (from the training data) information that the target-encoded means contain.

(e) Intuition for the results

Training frequency	Test rows	Baseline	Target Encoding	Embeddings	Combined
rare (≤ 5)	440	13.51	8.49	11.87	10.92
medium (6–50)	7,119	12.25	7.17	5.90	6.07
common (> 50)	52,130	12.68	7.13	6.22	6.42

Table 7: Test MAE by number of training rows for the test row's `StockCode`.

Training frequency	Test rows	Baseline	Target Encoding	Embeddings	Combined
rare (≤ 5)	398	34.68	28.66	25.34	25.15
medium (6–50)	11,420	11.94	7.49	6.16	6.27
common (> 50)	47,871	12.62	6.88	6.08	6.29

Table 8: Test MAE by number of training rows for the test row's `CustomerID`.

Intuitively, this type of problem is well-suited for an embedding layer. Consider that an embedding layer associates each category with a short list of learned numbers that serves as its "profile". The network then tunes these profiles to improve its predictions. Since retail data can contain features with thousands of categories, embedding gives a useful strategy for associating categories that behave alike — such as products bought in similar amounts or customers who order in similar ways — with similar profiles. The profile can hold more information than a single representative number (such as the average) for a category, and the network is also able to combine the stock-code and customer profiles. We see this complementarity paying off on the test set, where the embeddings reached much better accuracy than the target encoding.

To figure out where the gains in using embeddings come from, we can construct the MAE-by-frequency tables above. For categories seen more than five times in training, embeddings beat target encoding for both features. Rare categories, however, behave differently. For StockCode seen five or fewer times, embeddings are worse than target encoding. We can understand this in the framework we described above: profiles learned from few rows of data are unreliable, while target encoding's smoothing pulls these categories toward the overall mean. In these cases, we can expect the accuracy of an embedding model to worsen. For rare CustomerID, embeddings still perform better, but we should note that all models, including the baseline, do poorly in this

category. The combined model has no clear advantage on rare categories, but seems to perform slightly worse than embeddings alone on medium and common categories. We can justify this by recalling the seed results from part (d), which showed no clear gain from adding target encoding. Again, this is likely because an embedding network learns the information captured by target encoding along the way. Overall, rare categories comprise less than 1% of the test rows, so they barely affect the overall numbers.

Problem 3

(a) Data visualization

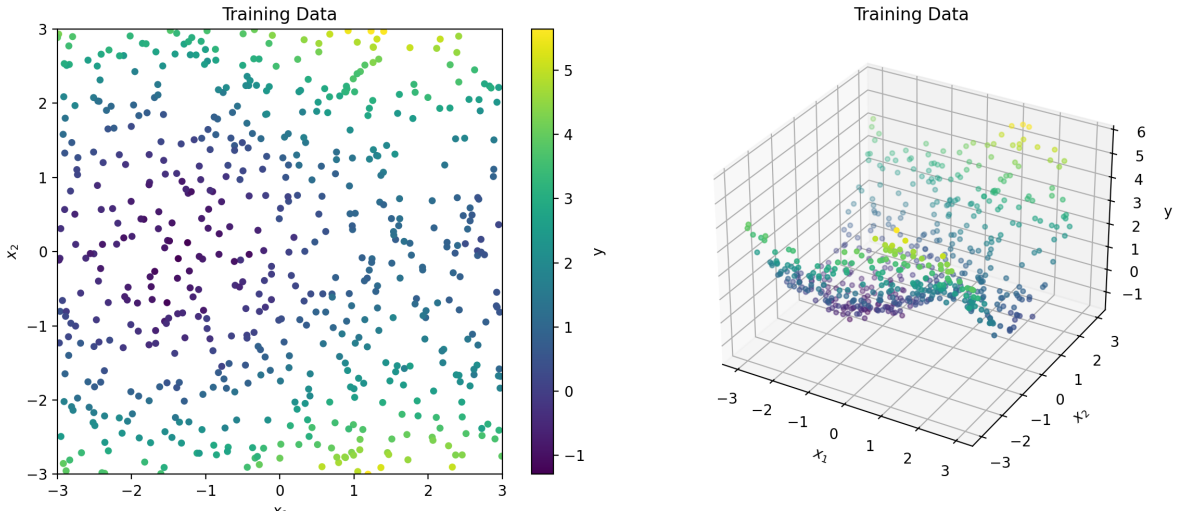


Figure 1: Training data: (x_1, x_2) colored by y (left) and y on the vertical axis (right).

(b) Random Kitchen Sinks with ridge regression

We implemented a Random Kitchen Sink model by drawing $h = D/2$ random projections with weights W drawn i.i.d. from $\mathcal{N}(0, \sigma^{-2})$ and phases $b \sim U[0, 2\pi]$. Each input is mapped to the $D + 1$ features $[\sin(W^T x + b), \cos(W^T x + b), 1]$, and the linear weights are fit by ridge regression. We chose to vary the number of random features $D \in \{10, 20, 50, 100, 200, 500, 1000, 2000\}$ as our hyperparameter, keeping the kernel scale ($\sigma = 1$) and the ridge strength ($\lambda = 10^{-3}$) fixed. For each D, we fit on the training set and record the training and validation MSE.

We find that small D tends to underfit, but the training and validation MSE both fall sharply by $D = 50$. After that, the training MSE plateaus, decreasing only slightly through $D = 2000$. We see that the validation MSE reaches its minimum at $D = 50$ and then rises slowly for larger D. This increasing train–validation gap is likely an indicator of mild overfitting. By selecting on validation loss, we take $D = 50$ as our optimal hyperparameter, but note that the differences between $D = 50$–200 are small — indicating that the model is not very sensitive to D past an initial underfitting regime.

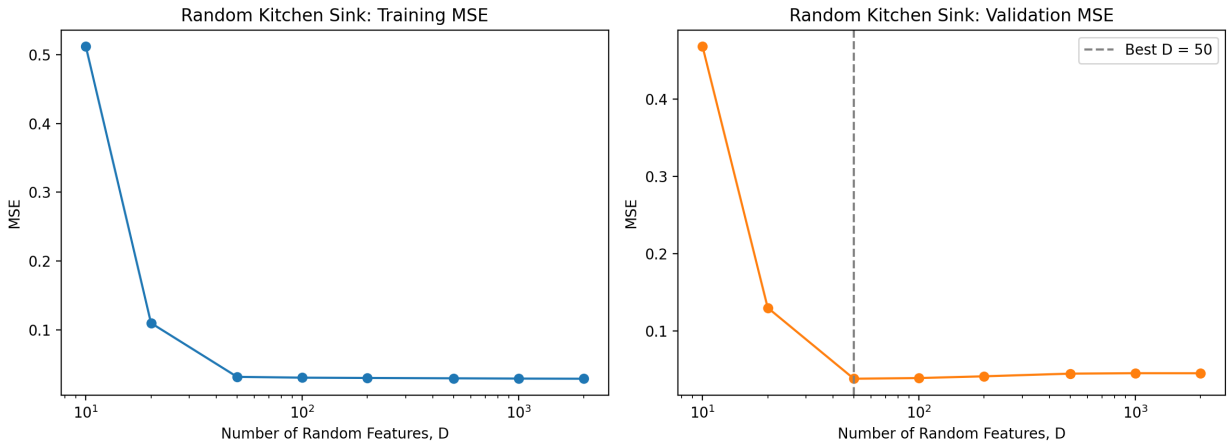


Figure 2: RKS training MSE (left) and validation MSE (right) as a function of the number of random features D. The dashed line marks the best D on validation.

D	Train MSE	Validation MSE
10	0.5122	0.4676
20	0.1096	0.1298
50	0.0313	0.0381
100	0.0301	0.0389
200	0.0297	0.0412
500	0.0292	0.0446
1,000	0.0288	0.0452
2,000	0.0286	0.0451

Table 9: RKS sweep over the number of random features D.

(c) One-hidden-layer MLP

We fit a one-hidden-layer MLP in PyTorch with a ReLU activation, so the network maps x through a linear layer of width `d_hidden`, then a ReLU, then a linear output layer. It is trained by minimizing the MSE loss with mini-batch SGD (using a learning rate of 10^{-2}, batch size 128, 2000 epochs). We chose to vary the hidden layer dimension `d_hidden` $\in \{2, 4, 8, 16, 32, 64, 128, 256, 512\}$ as our hyperparameter, keeping the learning rate, batch size, number of epochs, and activation fixed. For each `d_hidden`, we fit on the training set and record the training and validation MSE.

We find that very small networks tend to underfit, but the training and validation MSE both fall sharply by `d_hidden` $= 8$. Beyond that, the training MSE keeps decreasing slowly while the validation MSE plateaus. The validation MSE reaches its minimum at `d_hidden` $= 256$ and is essentially unchanged for higher `d_hidden`. By selecting on validation loss, we take `d_hidden` $= 256$ as the optimal hyperparameter, but note that the differences in validation MSE for `d_hidden` ≥ 16 are small — indicating that the model is not very sensitive to `d_hidden` once it has enough capacity. Unlike RKS, the validation curve does not rise noticeably for larger `d_hidden`, so there is no evidence of overfitting.

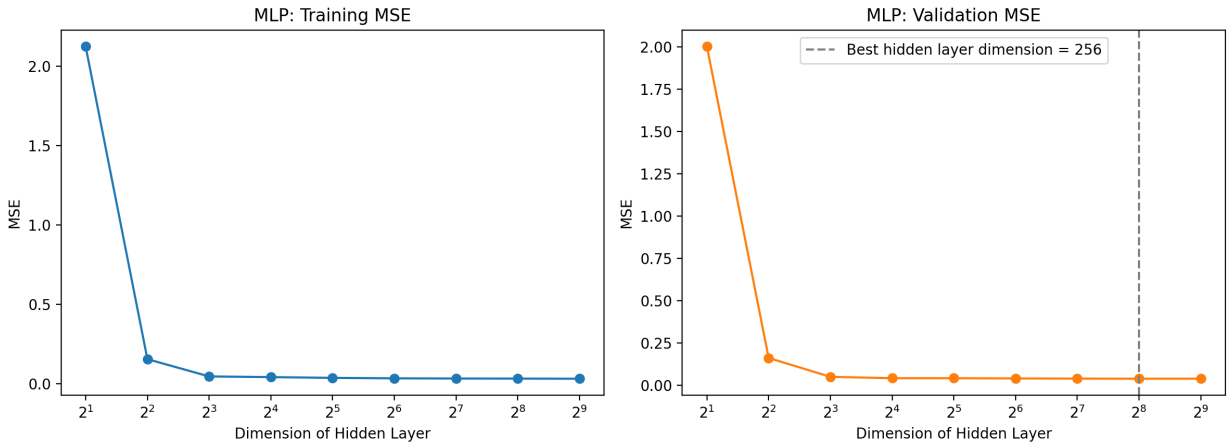


Figure 3: MLP training MSE (left) and validation MSE (right) as a function of the hidden layer dimension. The dashed line marks the best dimension on validation.

d_{hidden}	Train MSE	Validation MSE
2	2.1231	2.0020
4	0.1549	0.1622
8	0.0462	0.0508
16	0.0419	0.0426
32	0.0370	0.0425
64	0.0342	0.0412
128	0.0329	0.0403
256	0.0325	0.0396
512	0.0317	0.0399

Table 10: MLP sweep over the hidden layer dimension.

(d) Comparison and visualization

After refitting both models with the selected hyperparameters, RKS ($D = 50$) reaches a validation MSE of 0.0381, and the MLP (`d_hidden` $= 256$) reaches 0.0396. Although RKS performs slightly better, the difference is small and each number comes from a single random initialization, so it is reasonable to conclude that the two models are essentially comparable on this data set. However, RKS needed only 50 random sinusoidal features and a closed-form ridge solve, while the MLP needed a hidden layer five times wider and 2000 epochs of SGD just to reach a similar error. This difference in complexity is consistent with the data: the underlying function looks smooth, low-frequency, and has just two inputs, suggesting that sinusoidal features are a good match. The MLP, on the other hand, has to learn its features by stochastic gradient descent, a less direct method that also depends on optimization settings. We would expect an MLP to have the advantage on less smooth or higher-dimensional problems, but that does not apply to this data set. We save the predictions for the unlabeled `test.csv` in `rks_test_predictions.csv` and `mlp_test_predictions.csv`, one row per test point in the original order.

The training data in part (a) (Figure 1) shows a valley near $(x_1, x_2) \approx (-1.5, 0)$, where y is about -1. Away from it, y rises, with the highest values (about 5) near $(x_1, x_2) \approx (1.5, \pm 3)$ at the top and

Model	Best hyperparameter	Validation MSE
RKS (ridge)	$D = 50$	0.0381
MLP (1 hidden layer)	$d_{\text{hidden}} = 256$	0.0396

Table 11: Validation MSE of the two models refit with their best hyperparameters.

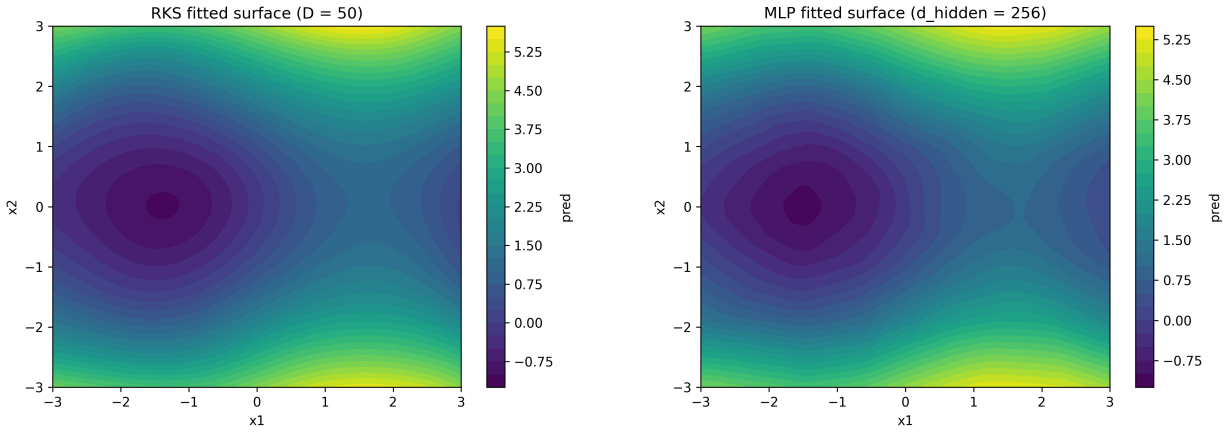


Figure 4: Fitted surfaces over $[-3, 3] \times [-3, 3]$ for the RKS model (left) and the MLP (right), each with its best hyperparameter.

bottom edges. Both fitted surfaces (Figure 4) reproduce this structure: a smooth basin centered near the same location, values increasing toward the top and bottom edges, and a saddle near $(x_1, x_2) \approx (1.5, 0)$ separating the valley from a second shallow dip along the right edge. The two surfaces are very similar in range and shape, which is consistent with their near-identical validation MSE. They differ mainly in smoothness. The RKS surface is smoother because it is built from a sum of sinusoids, while the MLP contours are slightly rougher. That jaggedness is the piecewise-linear signature of a ReLU network, which builds the surface from many linear pieces.